



1 - Uno sguardo di insieme alle applicazioni web

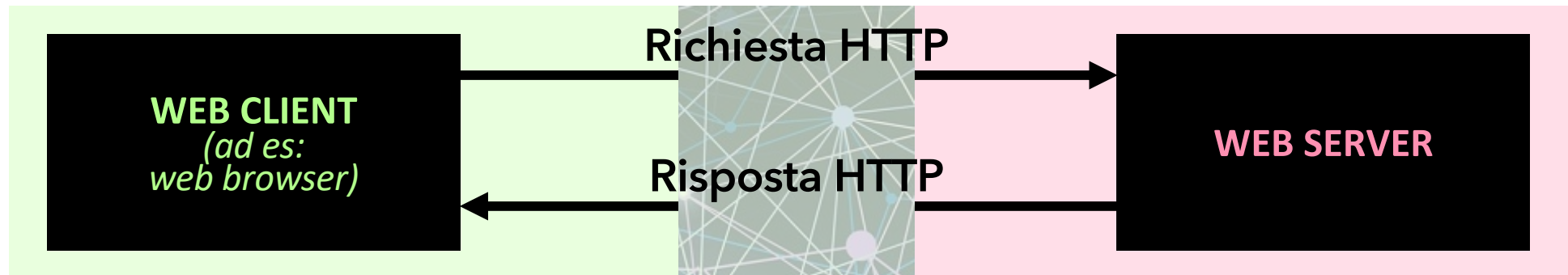
Il World Wide Web

WWW e HTTP

World Wide Web (WWW), detto anche in breve "Web"

è un servizio della rete Internet

- di tipo client/server
- basato sul protocollo di livello applicativo HTTP
- orientato alla pubblicazione e condivisione di contenuti e risorse digitali



il Web in origine

il Web nasce per la condivisione di documenti e media digitali (immagini, audio, video): l'obiettivo principale di una richiesta HTTP è dunque quello di specificare quale risorsa si voleva ottenere da un determinato Web Server; l'obiettivo principale della risposta HTTP è di "contenere" la risorsa richiesta corredata di meta-informazioni che permettessero al Web Client di riprodurla al meglio.

La Richiesta HTTP

In prima battuta, possiamo distinguere due tipologie di richieste. Le più comuni hanno l'obiettivo di ottenere (GET) una risorsa; devono dunque poter indicare univocamente qual è la risorsa in questione e su quale Web Server si trova. Deve anche specificare l'origine (o mittente) in modo che il Web Server possa inviare la risorsa al client che ha effettuato la richiesta stessa.

La seconda tipologia di richieste ha l'obiettivo di inviare dati (POST) al Web Server perché li elabori o li conservi in qualche modo; anche in questo caso la richiesta deve poter indicare la destinazione dei dati. Tale destinazione è sempre una "risorsa" del Web Server ma dovrà trattarsi di un tipo di risorsa in grado di effettuare una computazione, quindi uno script o un applicativo che il Web Server è in grado di eseguire, atto a ricevere proprio quei dati che il client sta inviando.

La Risposta HTTP

La Risposta HTTP deve innanzitutto indicare se la richiesta è andata a buon fine. Se la richiesta ha avuto successo, quasi sicuramente la risposta contiene dei dati: la risorsa desiderata nel caso di richieste GET, il risultato dell'elaborazione nel caso di richieste POST. Sarà compito della risposta indicare al client il tipo dei dati inviati in modo che esso possa renderli fruibili all'utente nel modo più opportuno.

individuare le risorse nel WWW

il sistema di catalogazione delle risorse nel Web

URL (Uniform Resource Locator)

una stringa di testo con un formato ben definito, in grado di identificare univocamente ogni risorsa disponibile sul Web

`protocol://hostname[:port]/path[?querystring][#fragment]`

`http://www.organizzazione.it:80/utenti/profilo?name=picardi#phone`

Struttura di una URL – I

Protocol

indica il protocollo di **livello applicativo** a cui la URL fa riferimento. Abbiamo detto che il Web è basato sul protocollo HTTP, ma le URL sono un formato utilizzato anche da altri servizi basati su altri protocolli. Quindi ad esempio il protocollo potrebbe essere FTP (File Transfer Protocol, per il download/upload di file su un server) o SMTP (Simple Mail Transfer Protocol, per l'invio di email), o altro.

Domain/Host

identifica lo specifico Web Server (**host** nel contesto dei protocolli di rete) a cui la richiesta è indirizzata. Tipicamente si compone di tre parti:

- il **dominio di terzo livello** o **sottodominio** (nell'esempio "www"), tipicamente associato a uno specifico host
- il **dominio di secondo livello** (nell'esempio "organizzazione", tipicamente associato a una rete locale di un'organizzazione, ente o azienda)
- il **dominio di primo livello** (nell'esempio "it") che identifica l'area geografica e/o la tipologia di organizzazione a cui il dominio di secondo livello fa riferimento

Il termine generico "dominio" di solito si riferisce alla combinazione di secondo e primo livello (quindi ad esempio "organizzazione.it"). I domini vengono acquistati da specifici provider e registrati, per garantirne l'univocità, presso il Registro di riferimento per il dominio di primo livello che si è scelto di utilizzare.

Port

è possibile che su uno stesso host rispondano più servizi; a ciascuno viene assegnato uno specifico "canale" detto **porta**, identificato da un numero. Nella normale navigazione su Web la porta non viene specificata perché è quella di default per il protocollo HTTP (80) o HTTPS (443) (HTTPS è la versione "sicura", in quanto cifrata, del protocollo HTTP).

Struttura di una URL – II

Path

Il path è il percorso per reperire sul server la risorsa; vedremo che può denotare un effettivo percorso di cartelle e sottocartelle che porta ad un **file** (in tal caso, l'ultimo elemento del percorso sarà il **nome del file** stesso), ma può anche denotare un percorso **logico** per identificare una **funzione** di un applicativo che risponde ad una richiesta di elaborazione.

I segmenti del percorso sono separati dal carattere di slash /.

Nell'esempio della slide precedente il path è composto da due segmenti: *utenti* e *profilo*

Query string

Si tratta di un testo opzionale che aggiunge ulteriori informazioni alla richiesta, sotto forma di **coppie chiave (c)/valore (v)**. La query string inizia sempre con il carattere **?** seguito dalle coppie **c=v** separate, se ce ne fosse più di una, da un **&**. Quindi ad esempio una query string può avere la forma:

`?min=5&max=100&sort=alphabetical`

Nell'esempio della slide precedente la query string è **?name=picardi**

Fragment

Quando la risorsa inviata in risposta è una pagina web scritta in linguaggio HTML, è possibile indicare tramite questa porzione della URL una collocazione precisa all'interno della pagina, a cui il browser deve posizionarsi una volta ottenuta la pagina stessa.

La pagina può definire al suo interno più **location**, ciascuna con un nome, e la URL può specificare a quale ci si vuole posizionare, facendone precedere il nome dal simbolo **#**.

Nell'esempio della slide precedente si suppone che la risposta alla richiesta sia una pagina in HTML contenente una location *phone* a cui ci si vuole posizionare; il fragment è dunque *#phone*.

Domain Name System

Trovare l'indirizzo IP di un host

URL (Uniform Resource Locator)

una stringa di testo con un formato ben definito, in grado di identificare univocamente ogni risorsa disponibile sul Web

`protocol://hostname[:port]/path[?querystring][#fragment]`

`http://www.organizzazione.it:80/utenti/profilo?name=picardi#phone`

DNS (Domain Name System)

il servizio di livello applicativo responsabile di tradurre lo hostname nel corrispondente indirizzo IP

- il protocollo HTTP è di livello applicativo e usa i servizi dei sottostanti protocolli TCP (livello trasporto) e IP (livello rete) che identificano gli host in rete tramite degli indirizzi numerici detti indirizzi IP
- prima di inviare una richiesta HTTP con una data URL, il client (browser) richiede al DNS server l'indirizzo IP corrispondente allo hostname
- solo a quel punto la richiesta HTTP può effettivamente partire

il linguaggio HTML

una rete di documenti collegati fra di loro

HTML (HyperText Markup Language)

il mattone fondamentale del Web sono le pagine HTML.

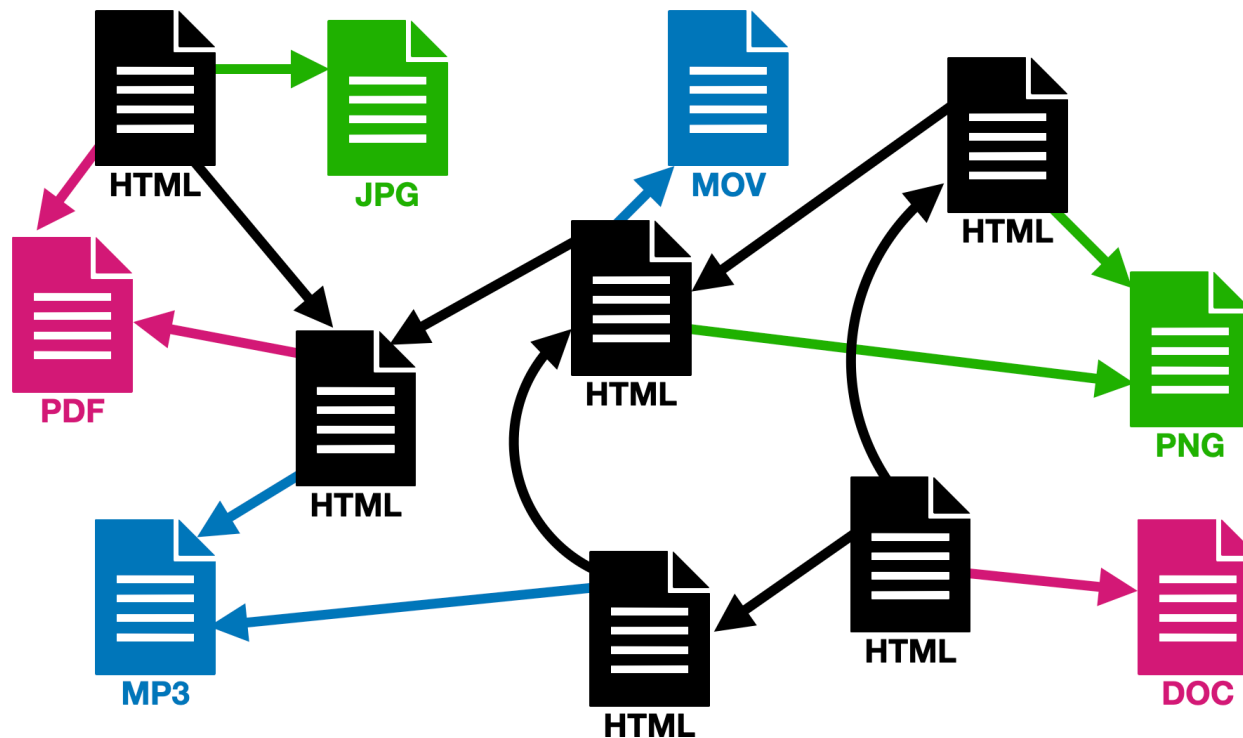
- documenti contenenti testo e media di varia natura, che il browser è in grado di visualizzare all'utente
- tramite il linguaggio HTML specificano al loro interno collegamenti (link) ad altre risorse
 - media o altre informazioni che vengono utilizzate dal browser per visualizzare quella stessa pagina
 - rimandi ipertestuali ad altre pagine HTML presenti nel Web
- è proprio grazie a tali collegamenti che le risorse presenti sul Web sono navigabili, è cioè possibile per l'utente spostarsi dall'una all'altra cliccando sui collegamenti che ciascuna fornisce.
- L'immagine delle risorse sparse per il mondo collegate l'una all'altra richiama una Ragnatela; a ciò il Web deve il suo nome.

Web, Applicazioni Web, Tecnologie Web...

i concetti chiave: HTML

HTML (HyperText Markup Language)

il mattone fondamentale del Web sono le pagine HTML.



L'Ipertesto

Già prima dell'esistenza del Web, Internet veniva usata per scambiare **risorse** (dati, grafici, articoli), soprattutto in ambito scientifico. La modalità con cui questo avveniva era tuttavia piuttosto complicata per gli utenti: bisognava conoscere lo **host name** che ospitava la risorsa, collegarsi, e scorrerlo cartella per cartella alla ricerca di ciò che interessava, con l'ulteriore difficoltà che ogni host organizzava i documenti al proprio interno secondo criteri di sua definizione.

Ogni volta che un articolo (all'epoca puramente testuale) aveva necessità di riferirsi ad un altro articolo, o a un file contenente dati sperimentali, o a un grafico, per fare alcuni esempi, l'utente doveva far ripartire da capo il processo di ricerca manuale, scoprendo quale host ospitava questa seconda risorsa e scartabellando per trovarla.

Il Web nasce, in questo contesto, alla fine degli anni 80 del ventesimo secolo grazie a due intuizioni:

- l'introduzione delle **URL**, che permettevano di avere un indirizzo univoco che facilitasse il reperimento di qualsiasi risorsa in modo uniforme, indipendentemente dalla sua collocazione
- far sì che i documenti condivisi, soprattutto gli articoli, potessero essere **ipertestuali** e **multimediali**

Un ipertesto è di base un **testo multidimensionale**, ossia in cui non c'è un'unica sequenza di lettura obbligatoria, ma nel quale ci possono essere salti, rimandi ad altre parti del testo o ad altri testi, e il fruitore è libero di muoversi fra di essi come preferisce.

Questo, in combinazione con la multimedialità, ossia la possibilità di rappresentare immagini, audio e video direttamente all'interno della visualizzazione del documento, semplificò grandemente la condivisione di documenti e risorse... con i risultati rivoluzionari che oggi sono sotto gli occhi di tutti [al punto che chi è nato dopo il 2000 neanche si rende conto di come poteva funzionare Internet – o la vita! – senza il Web].

HTML

HTML è un linguaggio informatico ma non è un linguaggio di programmazione, ossia non istruisce un agente di calcolo su come elaborare dati.

È, come dice il suo nome, un **linguaggio di markup**, ossia di annotazione.

Immaginiamo di avere scritto un articolo avendo solo la possibilità di scrivere *plain text* (ossia, testo senza alcun tipo di formattazione) e di dover comunicare a qualcun altro, che invece dispone di un software di word processing, come vogliamo che il nostro articolo sia strutturato, quali sono i titoli dei capitoli e quali dei paragrafi, dove sono le immagini, se ci sono intestazioni o pié di pagina, se il documento si divide in sezioni diverse, e così via.

Avendo a disposizione solo il *plain text* quello che possiamo fare è inserire nel nostro articolo delle etichette, che si distinguano in qualche modo dal testo vero e proprio, e che dicano: "questa riga è il titolo di un capitolo" oppure "questa scritta è una nota a pié di pagina", oppure ancora "questi due brani sono di due sezioni diverse".

Questo è esattamente ciò che fa HTML (in maniera un po' più codificata di così, ma il senso è quello). Le pagine HTML sono documenti *plain text* contenenti **tag** (etichette) che indicano al browser come strutturare la pagina. Ulteriori tag permettono inoltre di specificare **link ipertestuali**, che portano ad altre pagine, attivabili dall'utente cliccando su specifiche porzioni di testo o aree della pagina (**hot spot**).

HTML è complementato da un altro linguaggio, **CSS** (Cascading Style Sheet), tramite cui si comunica al browser con quale preciso aspetto grafico deve essere reso ciascun elemento (ad esempio "i titoli devono essere grandi 20 punti e blu", "lo sfondo deve essere nero e le scritte bianche", "le immagini devono essere allineate al centro con un bordo nero", e così via).

HTML e CSS saranno oggetto di successivi approfondimenti.

esecuzione di programmi nel web server

la possibilità di restituire risorse calcolate sul momento

risorse/pagine statiche

una risorsa (o pagina, se si tratta di un documento HTML) individuata da una URL è statica se corrisponde a un file già esistente, che deve semplicemente essere restituito dal server al client

- in questo caso la URL rimanda effettivamente a una risorsa fisica (anche se non è detto che il path logico espresso nella URL corrisponda al path all'interno del file system del server)

risorse/pagine dinamiche

una risorsa (o pagina, se si tratta di un HTML) individuata da una URL è dinamica se viene calcolata sul momento dal Web Server

- in questo caso la URL rimanda ad un programma o script eseguibile il quale ha la responsabilità di effettuare un calcolo e restituire al client il risultato (sulla base di dati contenuti nella richiesta HTTP e/o di dati presenti nel file system del web server e/o di basi di dati e/o di dati forniti da servizi di terze parti)

Esecuzione di programmi lato server

Un Web Server è un software (open source o commerciale) che viene installato sulla macchina che fa da host.

Diverse tipologie di Web Server sono in grado (eventualmente installando opportune estensioni) di eseguire programmi in diversi linguaggi di programmazione, tipicamente linguaggi interpretati (come JavaScript) o ibridi (come Java), contenendo al proprio interno un agente di esecuzione per quel dato linguaggio (ad es, una Java Virtual Machine).

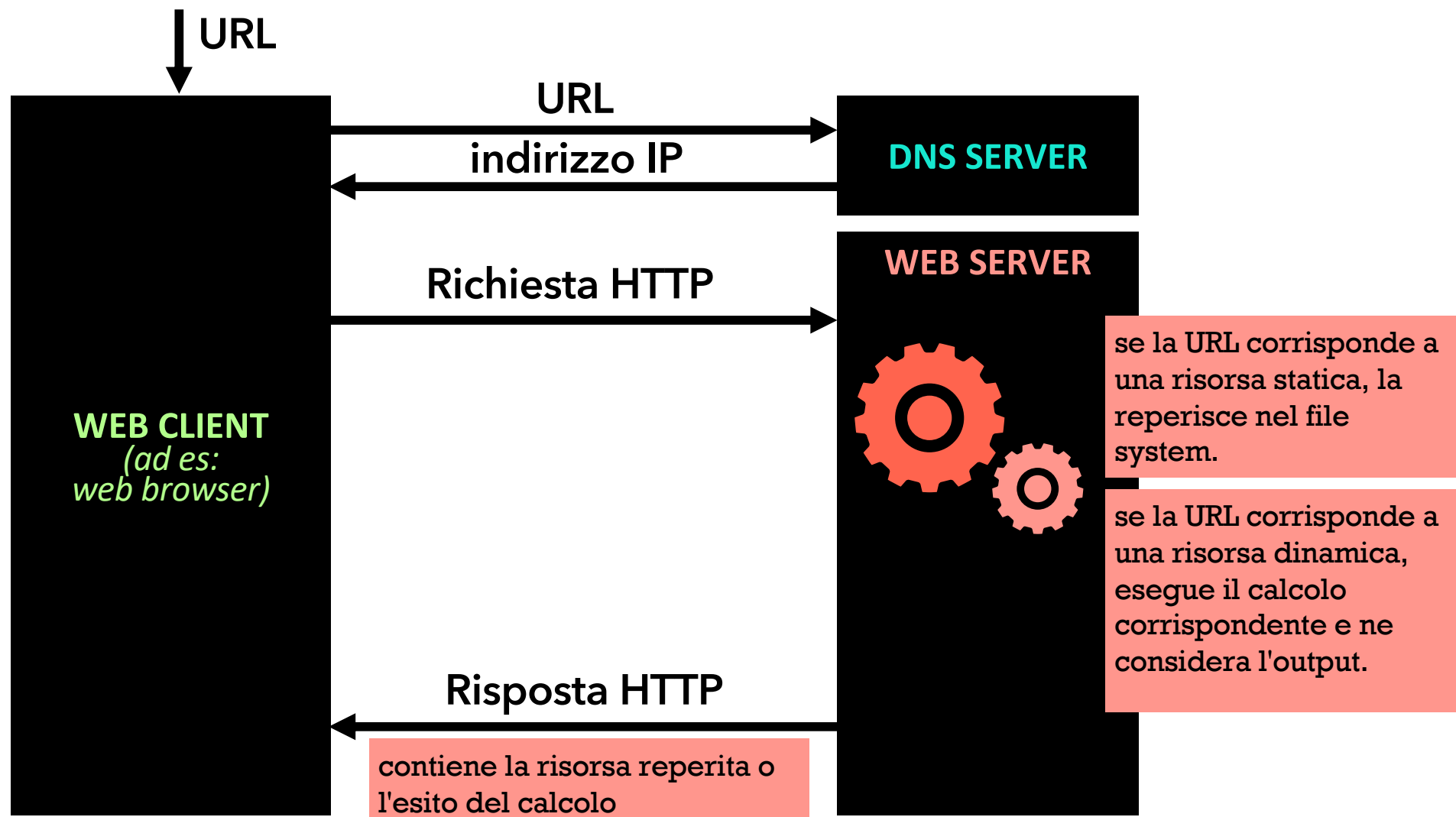
Questo fa sì che sia possibile scrivere veri e propri applicativi (detti server-side perché vengono appunto eseguiti all'interno del Web Server) che rispondono a determinate richieste HTTP su determinate URL, attivando in risposta opportune funzioni di calcolo. La specifica funzione di calcolo attivata dipenderà dal path, dall'eventuale query string, e dai dati contenuti nella richiesta stessa se si tratta di una richiesta di tipo POST. Questi stessi elementi possono altresì giocare il ruolo di input del calcolo stesso.

Dopo aver determinato l'output del calcolo, un applicativo server-side costruisce una risposta sulla base di esso, e la incapsula in una risposta HTTP da restituire al client.

Nel laboratorio di questo corso vedremo applicativi server-side scritti in JavaScript e in Java, ma altri linguaggi diffusi per applicazioni server-side sono C#, Php, Python, Ruby...

Anatomia di una transazione HTTP

richiesta + elaborazione + risposta



Anatomia di una transazione HTTP – I

1. il client (**browser**) "riceve" una URL

La URL può essere fornita dall'utente in modo **diretto** (la scrive nella barra degli indirizzi del browser) o **indiretto** (clicca su un link ipertestuale presente nella pagina che sta visualizzando).

Può anche essere **l'effetto di una precedente richiesta**: se il browser riceve una pagina HTML che deve essere visualizzata, è possibile che quella pagina richieda altre risorse (ad esempio, immagini o altri *media*) che corrispondono ad altrettante URL. Per poter visualizzare la pagina ottenuta tramite una prima richiesta HTTP, il browser deve dunque prima soddisfare altre N richieste HTTP.

C'è una terza possibilità che vedremo meglio a breve: se la pagina HTML visualizzata è la **user interface** (UI) di una applicazione *client-side*, è possibile che la URL venga generata come **effetto dell'esecuzione di codice lato client**.

2. il client richiede al **DNS** l'**indirizzo IP** dell'host specificato nella URL

3. il client **invia la richiesta** HTTP all'indirizzo IP in questione

l'indirizzo IP serve per inviare la richiesta; all'interno della richiesta però sarà presente la URL originale. Vedremo meglio in seguito la struttura specifica delle richieste HTTP.

4. quando la richiesta giunge al server, un primo smistamento avviene grazie al numero di **porta**

su uno stesso host ci possono essere diversi servizi che rispondono su diverse porte, l'host dovrà quindi verificare che la porta specificata corrisponda effettivamente a un web server e non a qualche altro tipo di servizio!

Anatomia di una transazione HTTP – II

5. **il Web Server esamina la URL, in particolare il path, e stabilisce se quanto richiesto corrisponde a una risorsa esistente, e in tal caso, se si tratta di una risorsa statica o dinamica**
 - a. se la risorsa richiesta è statica, il Web Server confeziona la Risposta HTTP incapsulando al suo interno la risorsa così come l'ha reperita nel proprio file system
 - b. se la risorsa è dinamica, significa che la URL corrisponde a una funzione eseguibile di un programma server-side; il programma verrà eseguito e l'output verrà incapsulato nella risposta. Per il calcolo vengono utilizzate tutte le informazioni presenti nella Richiesta HTTP, inclusa la struttura della URL, gli elementi che compongono il path, e l'eventuale query string.

Non è detto che lo scopo della funzione eseguita sia ottenere un output da inviare in risposta; lo scopo potrebbe essere ad esempio di salvare un dato in un DB o di comunicare un'informazione a un servizio di terze parti. Ciononostante l'esecuzione avrà un esito (anche solo il fatto che sia stata completata correttamente) che deve essere comunicato dal Web Server all'interno della Risposta HTTP.
 - c. se la risorsa non esiste su quel Web Server, esso confeziona ugualmente una Risposta HTTP, che però, come vedremo meglio in seguito, conterrà un codice di errore... a tutti sarà capitato di vedere apparire nel proprio browser la scritta 404 Not Found!
6. **La Risposta HTTP elaborata dal Web Server viene spedita indietro al client**

Il client (browser) "spacchetta" la risposta, determinando innanzitutto il tipo di dato che ha ricevuto. Se è una pagina HTML, la visualizza. Il browser è solitamente in grado di visualizzare autonomamente anche altri tipi di file, come immagini, audio, video, o documenti PDF. Se si tratta di un tipo di dato non gestito dal browser, verrà proposto all'utente di scaricarlo come file sul proprio disco.

esecuzione di programmi nel browser

rendere le pagine html interattive

pagine html passive

una pagina HTML è passiva quando può solo essere guardata/letta

- l'utente non può interagire in alcun modo con la pagina, se non al limite per cliccare su un link ipertestuale e spostarsi dunque in un'altra pagina (ciò che comunemente chiamiamo navigare)

pagine html interattive

una pagina HTML è interattiva quando il browser "fa qualcosa" (che non sia navigare) in risposta ad azioni dell'utente

- questo "fare qualcosa" concretamente corrisponde all'esecuzione di codice
 - una concezione corretta dell'interattività prevede che oltre a "fare qualcosa" l'aspetto della pagina si modifichi per dare conto all'utente che "qualcosa è stato fatto"
- interattività semplice: l'effetto dell'azione dell'utente è solo sulla visualizzazione, tipicamente volto a migliorare la fruizione della pagina
- interattività complessa: l'effetto dell'azione dell'utente è una vera e propria elaborazione dati e può prevedere la generazione di richieste verso un Web Server → si parla allora di applicazione client-side

Esecuzione di programmi lato client

Ogni browser contiene un motore di esecuzione per il linguaggio JavaScript, che è un **linguaggio interpretato**.

Quando al browser arriva una pagina HTML corredata da uno script JavaScript, il browser lo esegue.

L'effetto di tale esecuzione è principalmente quello di "popolare" l'ambiente di esecuzione di **funzioni** da eseguire in risposta a ben precisi eventi, tipicamente azioni dell'utente (click o spostamenti del mouse, compilazione di campi di testo, scorrimento della pagina, ecc). L'ambiente di esecuzione mette a disposizione di tali funzioni una rappresentazione ad oggetti (che discuteremo successivamente) della pagina HTML, permettendo loro di modificarne la struttura e quindi cambiare l'aspetto della pagina.

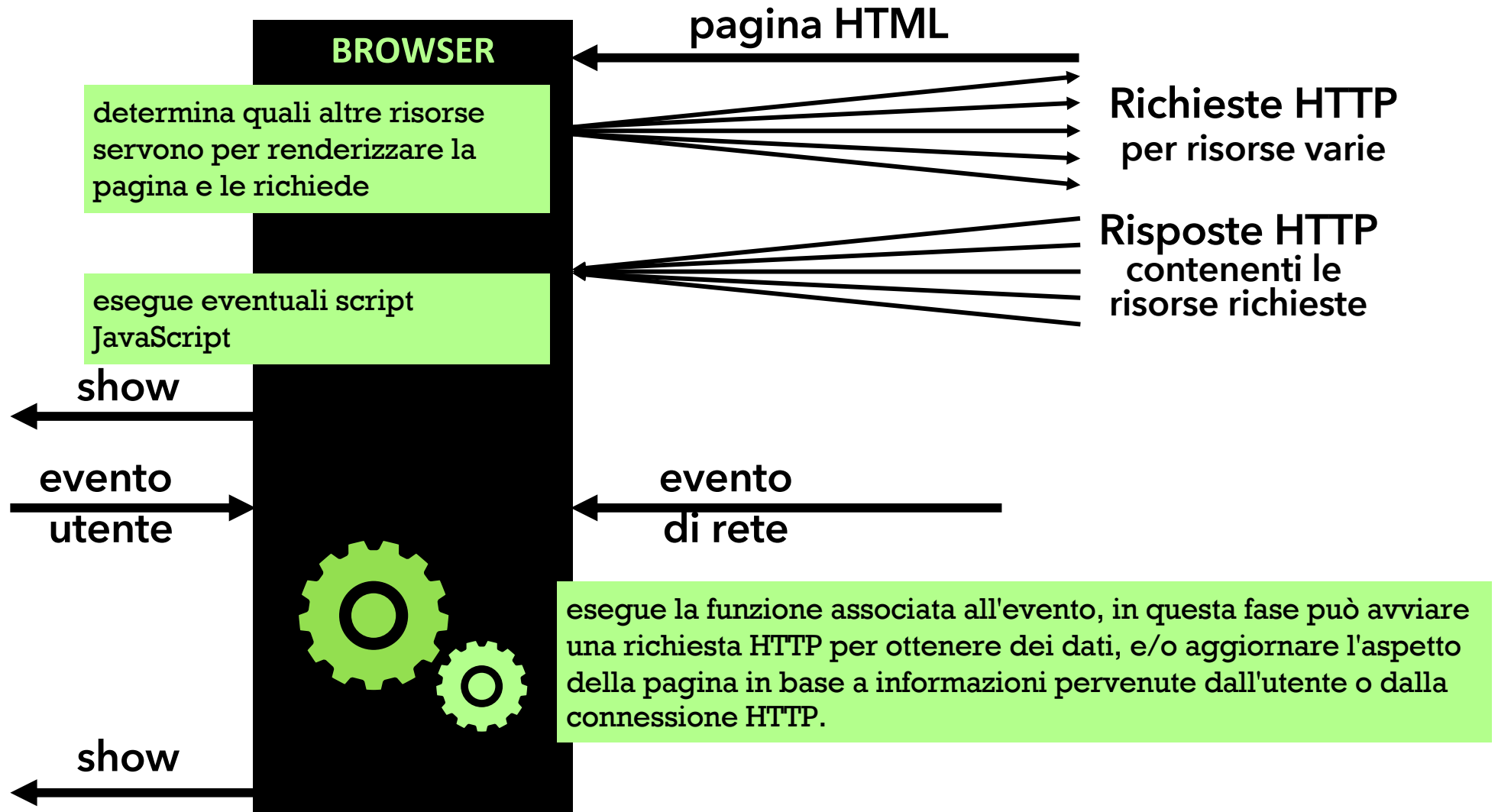
L'ambiente di esecuzione può anche venire popolato di **oggetti e variabili** che mantengono **uno stato di esecuzione**, il quale viene aggiornato in base alle azioni dell'utente; in questo modo la pagina HTML può trasformarsi a tutti gli effetti nella *user interface* di un applicativo, che in questo caso è detto "**client-side**" perché viene eseguito dal Web Client, ossia il browser.

Il codice eseguito *client-side* può, fra le altre cose, effettuare delle Richieste HTTP. Le Richieste HTTP formulate dal codice JavaScript hanno una peculiarità: i dati contenuti nella risposta non vengono gestiti dal browser (che, come abbiamo visto, può solo visualizzarli, se ne è in grado, o chiedere all'utente di scaricarli); vengono piuttosto ricevuti dallo stesso codice che ha formulato la richiesta. Esso potrà così elaborarli ed utilizzarli per modificare opportunamente ciò che l'utente vede.

Il browser è in grado di eseguire solo JavaScript; le applicazioni client-side o sono scritte direttamente in JavaScript, o devono comunque essere "tradotte" in JavaScript prima di poter essere eseguite.

Il ruolo del browser

richiesta + elaborazione + risposta



Anatomia di un ciclo del browser – I

1. il client (browser) ottiene (come esito di un precedente transazione HTTP) una **pagina HTML**
2. il browser determina se la pagina HTML specifica ulteriori risorse necessarie ad una sua corretta rappresentazione grafica (detta anche **renderizzazione**).
Come vedremo, la maggior parte delle pagine HTML specifica come risorsa aggiuntiva uno o più **fogli di stile** (CSS), ossia insiemi di regole che istruiscono il browser sull'aspetto grafico dei vari elementi della pagina.
Possono poi esserci **immagini** o altri **media** che appaiono nella pagina.
Se si tratta di una **pagina interattiva**, fra le risorse aggiuntive ci saranno anche uno o più **script** JavaScript.
3. non appena ha le risorse necessarie alla visualizzazione, il browser **renderizza** la pagina per mostrarla all'utente
se la pagina è **passiva**, ossia non prevede l'esecuzione di script, il ciclo del browser **termina** qui.
4. non appena riceve gli eventuali script, il browser li esegue
in teoria i passi 3 e 4 potrebbero avvenire in qualsiasi ordine, a seconda di cosa arriva prima dalla rete. Ma nella maggior parte dei casi si fa in modo (e c'è il modo di farlo) che gli script vengano eseguiti solo quando tutto il resto della pagina è caricato.

Anatomia di un ciclo del browser – II

Come detto in precedenza, gli script predispongono l'ambiente di esecuzione, creando **oggetti** che mantengono lo stato dell'applicazione, e specificando quali **funzioni** devono essere eseguite in corrispondenza di quali **eventi**.

Ci sono due tipologie principali di eventi che vengono tipicamente gestiti in una applicazione client-side: gli **eventi utente** (click del mouse, pressione di tasti della tastiera, scrolling, tap del dito nel caso di dispositivi mobili...) e gli **eventi di rete**, principalmente dovuti all'arrivo della risposta di una richiesta HTTP precedentemente effettuata, o a *timeout* di mancata connessione.

5. **sopraggiunge un evento (utente o di rete)**

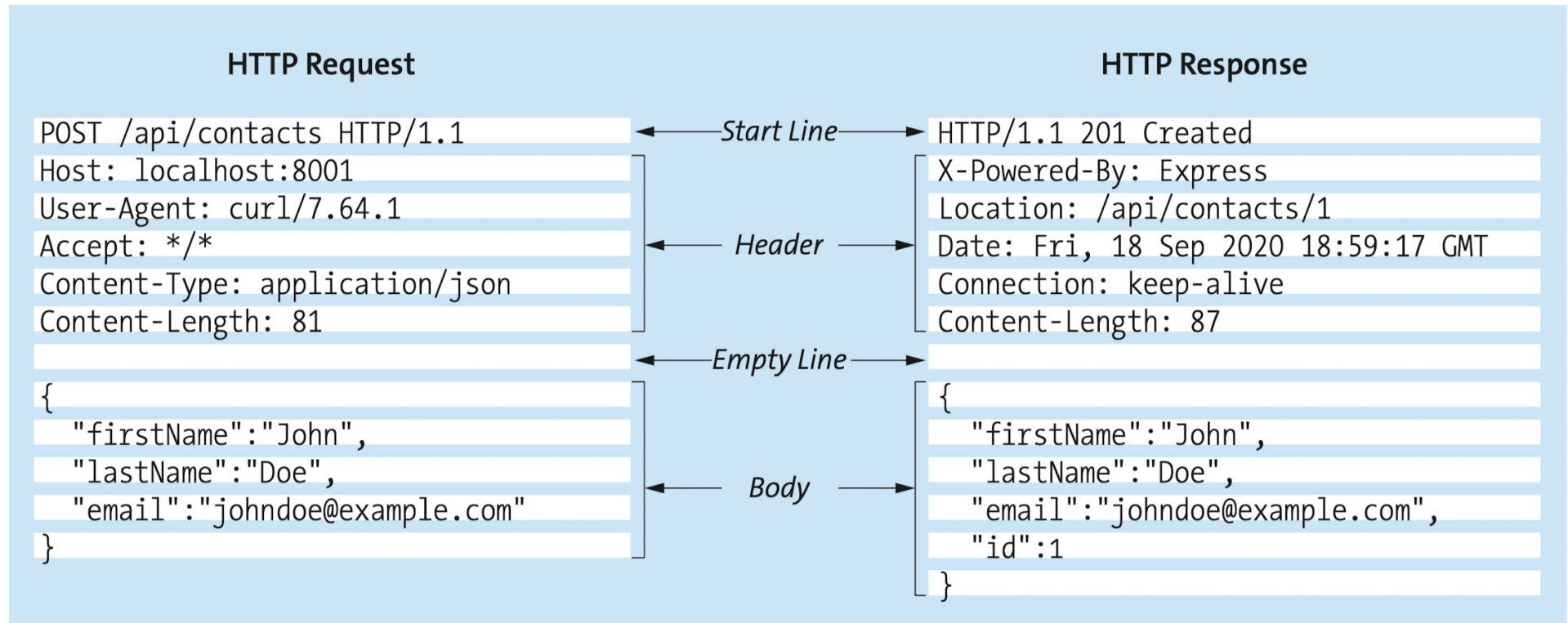
Se presente, il browser attiva la funzione precedentemente registrata per quell'evento. Tale funzione effettuerà elaborazioni, i cui effetti "percepibili" possono essere principalmente due: **generare altre connessioni HTTP** (per ottenere dei dati, o per spostarsi dalla pagina attuale), e **modificare l'aspetto della pagina** attualmente in visualizzazione.

Le richieste HTTP generate non sono bloccanti: vengono avviate, ma la gestione della futura risposta è demandata, in modo **asincrono**, ad una funzione che viene, come in precedenza, registrata presso l'ambiente di esecuzione per essere invocata all'arrivo della risposta HTTP.

6. **quando l'aspetto della pagina viene modificato, il browser la ri-renderizza.**

Il protocollo HTTP

struttura della comunicazione fra browser e Web Server



Fonte: Philip Ackermann, Full Stack Web Development. Rheinwerk Publishing (2023)

Struttura generale di HTTP Request e Response

Start line (linea iniziale)

Definisce la natura della richiesta o della risposta. Per questa ragione è chiamata anche **request line** o **response line**.

Header (intestazioni)

Dettagliano la richiesta/risposta. Sono dati sotto forma di coppie

header name: header value

L'elenco di *header name* possibili è piuttosto lungo, vedremo in seguito quelle più pertinenti per i nostri scopi.

Start line + *header* formano le *meta informazioni* della richiesta/risposta. Sono separati dalla parte successiva da una linea vuota (**empty line**).

Body (corpo)

Detto anche **payload** (letteralmente, «carico»), è il contenuto vero e proprio del messaggio. Nel caso di una richiesta non è detto che ci sia, perché alcune richieste sono volte ad **ottenere** dei dati dal server, non a **inviarne**.

Nel caso di una **Richiesta Http** può trattarsi ad esempio dei dati inseriti dall'utente in un form, o di dati elaborati dall'applicazione client-side in formato JSON (JavaScript Object Notation, la approfondiremo successivamente).

Nel caso di una **Risposta Http** potrà trattarsi di una pagina HTML, di un'immagine o altro *media*, di uno script JavaScript, di un qualsiasi altro tipo di file, o di dati elaborati in risposta da un'applicazione server-side, anche in questo caso solitamente in formato JSON.

La HTTP Request

Un po' più in dettaglio

HTTP Request

```
POST /api/contacts HTTP/1.1
Host: localhost:8001
User-Agent: curl/7.64.1
Accept: */*
Content-Type: application/json
Content-Length: 81

{
  "firstName": "John",
  "lastName": "Doe",
  "email": "johndoe@example.com"
}
```

Request line

- **HTTP method**
specifica quale **azione** deve essere eseguita sul server. Ci sono varie azioni (dette **method** o **verb**) HTTP possibili, per il momento ci considereremo i due «di base»:
 - **GET** – il server deve fornire informazioni al client
 - **POST** – il server deve elaborare in qualche modo i dati che sta ricevendo dal client
- **destinazione**
URL della richiesta (**completa** o con solo il **path assoluto** – si assume che lo hostname sia quello che sta ricevendo la richiesta)
- **versione**
la versione utilizzata del protocollo HTTP

La HTTP Request

Un po' più in dettaglio

HTTP Request

```
POST /api/contacts HTTP/1.1
Host: localhost:8001
User-Agent: curl/7.64.1
Accept: */*
Content-Type: application/json
Content-Length: 81

{
  "firstName": "John",
  "lastName": "Doe",
  "email": "johndoe@example.com"
}
```

Header

Molte coppie (name, value) sono impostate **di default** dal browser quando la richiesta viene formulata.

Più comunemente impostati dal programmatore:

- **Accept**
Tipi di dati accettati in **risposta**
- **Content-Type**
Tipo di dati contenuti nel **body**
- **User-Agent**
Informazioni sul tipo di **client** che sta formulando la richiesta

La HTTP Request

Un po' più in dettaglio

HTTP Request

```
POST /api/contacts HTTP/1.1
Host: localhost:8001
User-Agent: curl/7.64.1
Accept: */*
Content-Type: application/json
Content-Length: 81
```

```
{
  "firstName": "John",
  "lastName": "Doe",
  "email": "johndoe@example.com"
}
```

Tipi MIME

Negli **header HTTP** quando è necessario specificare il tipo di dato accettato o inviato si usa il formato **MIME** (Multipurpose Internet Email Extension) nati per specificare i tipi degli attachment nelle email

Il formato è **tipo/sottotipo**, alcuni tipi comuni:

text/html
text/plain
text/css
text/javascript
image/jpeg

image/png
application/json
application/pdf
application/xml
video/mp4

La HTTP Request

Un po' più in dettaglio

HTTP Request

```
POST /api/contacts HTTP/1.1
Host: localhost:8001
User-Agent: curl/7.64.1
Accept: */*
Content-Type: application/json
Content-Length: 81
```

```
{
  "firstName": "John",
  "lastName": "Doe",
  "email": "johndoe@example.com"
}
```

Body

Le richieste di tipo **GET** non hanno un **body** perché il loro obiettivo è **ottenere** dei dati, non inviarne.

Le richieste di tipo **POST** hanno come obiettivo **inviare** dati e dunque hanno un **body** tipicamente di uno dei seguenti tipi:

application/x-www-form-urlencoded

dati contenuti in un form compilato dall'utente e inviato dal browser

application/json

dati elaborati da una app client-side e inviati dall'app stessa

La HTTP Response

Un po' più in dettaglio

HTTP Response

```
HTTP/1.1 201 Created
X-Powered-By: Express
Location: /api/contacts/1
Date: Fri, 18 Sep 2020 18:59:17 GMT
Connection: keep-alive
Content-Length: 87

{
  "firstName": "John",
  "lastName": "Doe",
  "email": "johndoe@example.com",
  "id": 1
}
```

Response line

- **versione**
la versione utilizzata del protocollo HTTP
- **status code**
dice se la richiesta è stata elaborata con successo o meno; i possibili stati sono divisi in 5 categorie rappresentate dalla prima cifra del codice, in particolare:
 - 2XX – richiesta processata con successo
 - 4XX – richiesta fallita per un errore del client
 - 5XX – richiesta fallita per un errore del server
- **status text**
descrive testualmente lo stato
Es: status code = 404, status text = «Not found»

La HTTP Response

Un po' più in dettaglio

HTTP Response

```
HTTP/1.1 201 Created
X-Powered-By: Express
Location: /api/contacts/1
Date: Fri, 18 Sep 2020 18:59:17 GMT
Connection: keep-alive
Content-Length: 87

{
  "firstName": "John",
  "lastName": "Doe",
  "email": "johndoe@example.com",
  "id": 1
}
```

Header

Header più comuni per la risposta HTTP (anche qui, molti sono impostati in automatico dal Web Server):

- **Content-Type**
Tipo di dati contenuti nel **body**
- **Content-Length**
Lunghezza del **body** in bytes
- **Server**
Dettagli sulla tipologia di Web Server
- **Date**
Giorno e ora dell'invio della risposta

La HTTP Response

Un po' più in dettaglio

HTTP Response

```
HTTP/1.1 201 Created
X-Powered-By: Express
Location: /api/contacts/1
Date: Fri, 18 Sep 2020 18:59:17 GMT
Connection: keep-alive
Content-Length: 87
```

```
{
  "firstName": "John",
  "lastName": "Doe",
  "email": "johndoe@example.com",
  "id": 1
}
```

Body

Le risposte hanno **necessariamente** un **body**.

Tipi più comuni di **body** in una response:

text/html

la risposta è una pagina html che il browser visualizzerà

application/json

la risposta contiene dei dati che ci si aspetta vengano recepiti da un'applicazione client-side

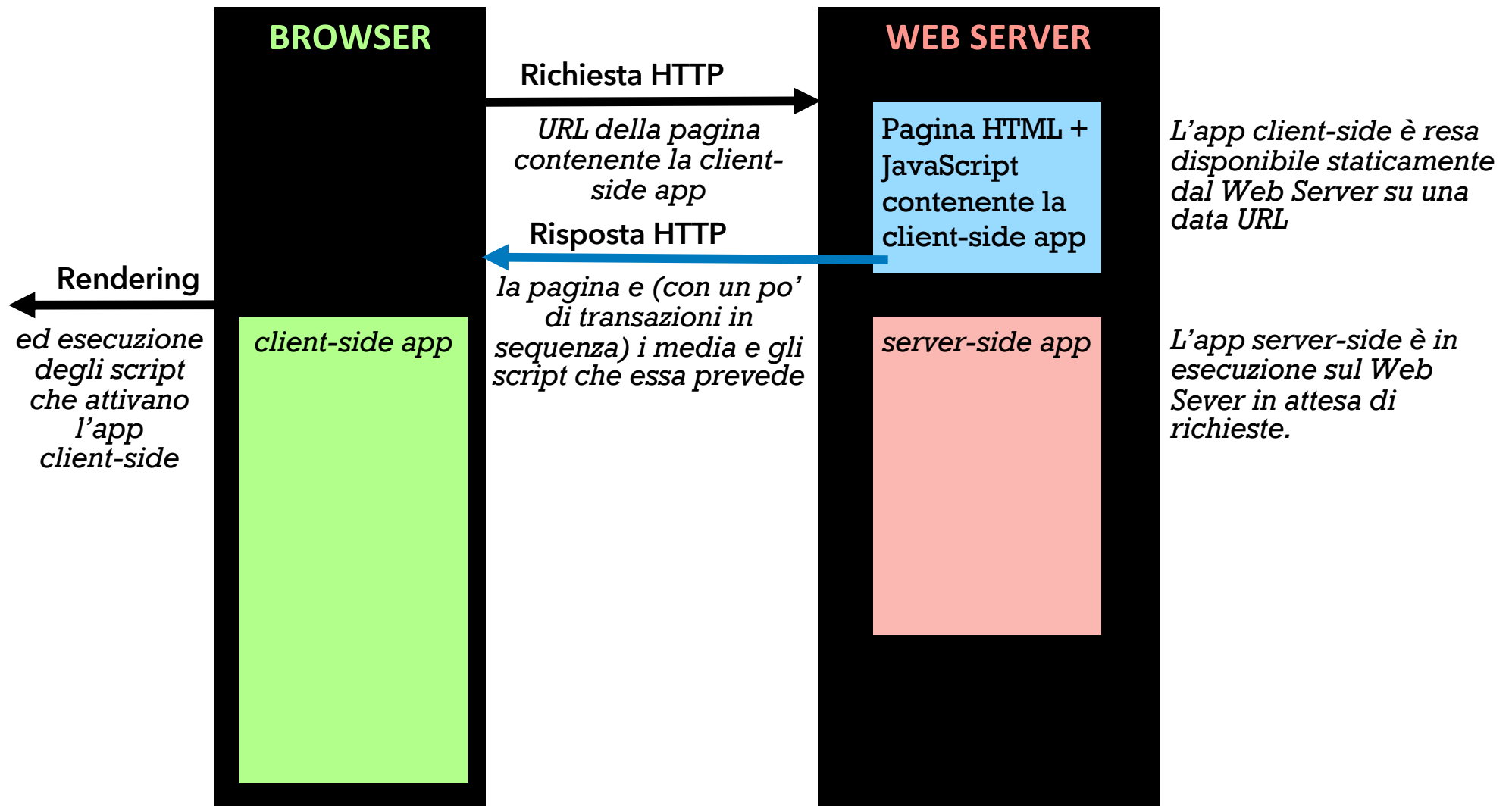
text/*, image/*, audio/*, video/*

application/*

fogli di stile, script JavaScript, immagini e altri file multimediali, file scaricabili...

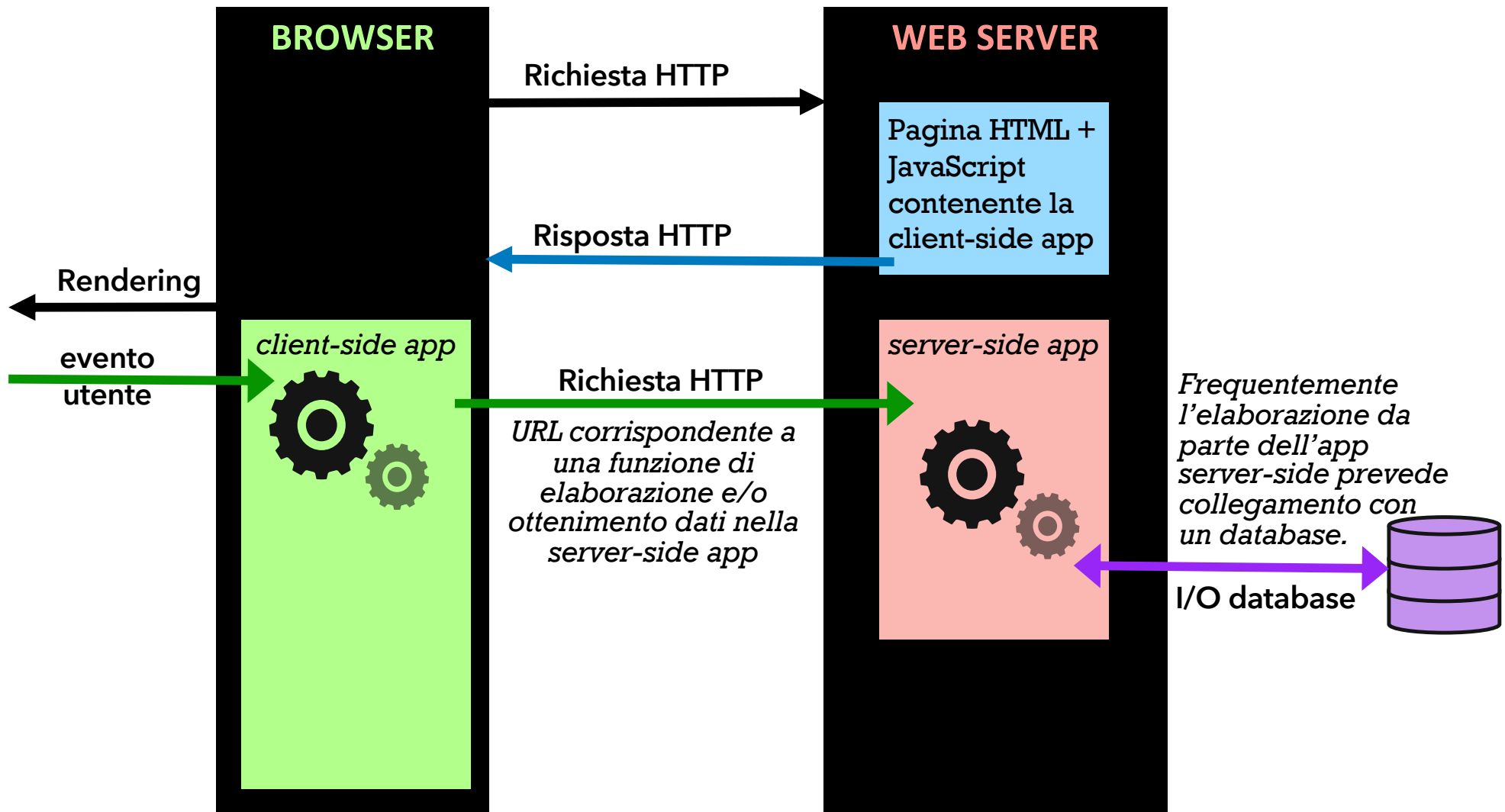
Applicazioni Web

Il caso più completo: *server-side* + *client-side*



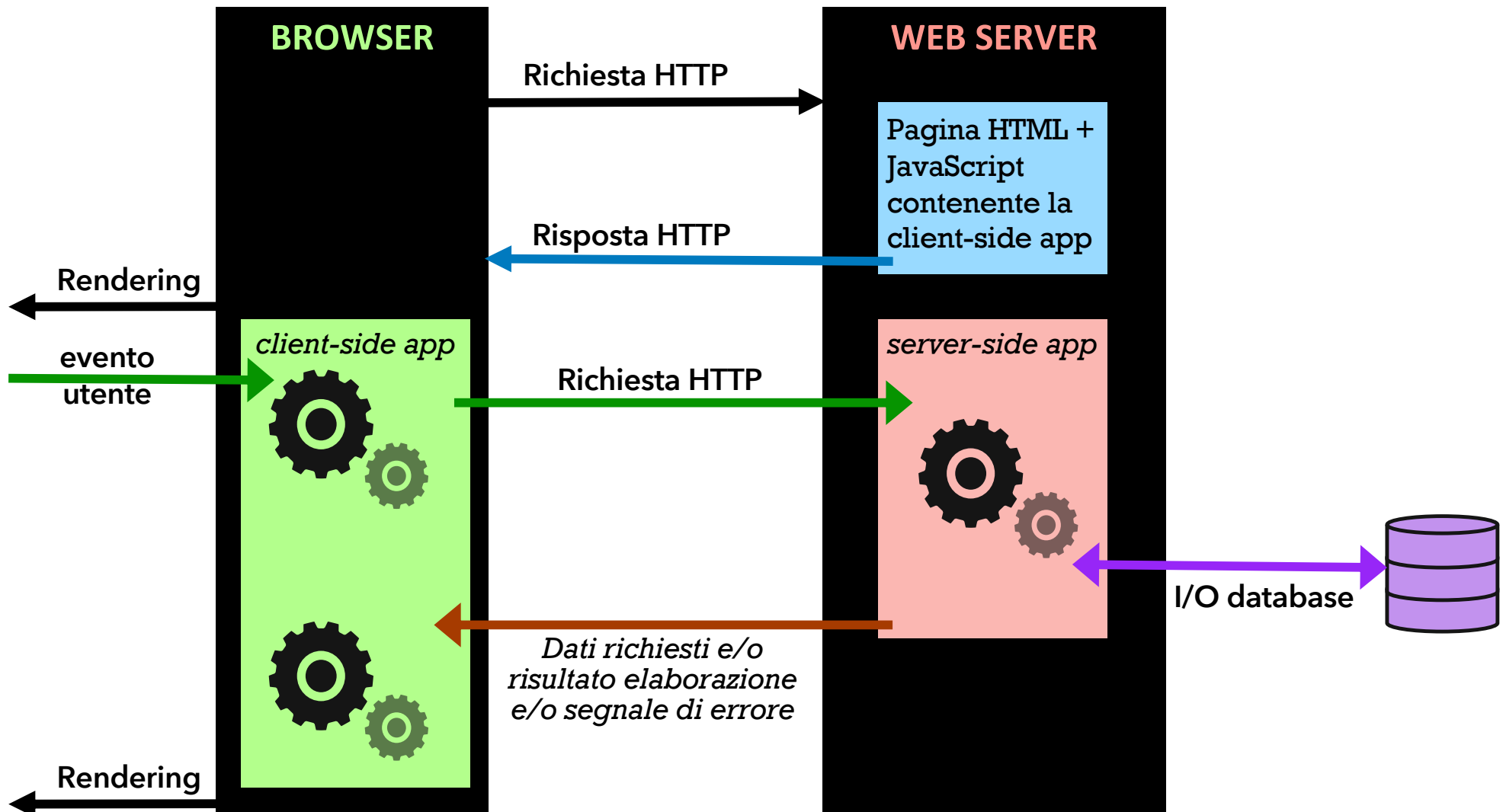
Applicazioni Web

Il caso più completo: *server-side* + *client-side*



Applicazioni Web

Il caso più completo: *server-side* + *client-side*



Esecuzione di un'app web client-side + server-side («full stack») - I

1. In linea di massima il **Web Server** è sempre online (salvo lavori di manutenzione). Il Web Server pubblica in modo statico la pagina HTML a cui sono collegati gli script che costituiscono l'applicazione client-side. Parallelamente, esso mantiene in esecuzione l'app server-side. Opportuni meccanismi del Web Server «bootstrappano» le applicazioni server-side installate nel momento in cui il Web Server viene avviato. Il Web Server è l'ambiente operativo in cui l'app server-side viene eseguita. Così come le applicazioni desktop vengono avviate da, e si interfacciano con, il Sistema Operativo, le applicazioni web server-side vengono avviate da, e si interfacciano con, il Web Server. L'applicazione server-side risponde su un dato insieme di URL (ciascuna corrispondente ad una diversa funzionalità). L'insieme di URL su cui un'applicazione server-side risponde viene anche detto **API** dell'applicazione server-side.
2. Il browser richiede la pagina HTML che corrisponde all'applicazione client-side. Una volta ottenuta la pagina HTML, come visto in precedenza parte una sequenza di transazioni volte ad ottenere i media presenti nella pagina e gli script ad essa collegati. A questo punto il browser fa due cose:
 - Renderizza la pagina, perché l'utente possa visualizzarla e interagire con essa
 - Attiva l'ambiente di esecuzione JavaScript ed esegue gli script; come abbiamo visto questo corrisponde all'avvio dell'applicazione client-side, che è costituita da un insieme di funzioni e oggetti di cui l'ambiente di esecuzione viene popolato. La memoria dinamica dell'ambiente di esecuzione non viene resettata finché la pagina rimane aperta nel browser. Ogni pagina del browser ha un proprio ambiente di esecuzione indipendente. Le applicazioni web client-side vengono avviate da, e si interfacciano con, il browser e l'ambiente di esecuzione JavaScript.

Esecuzione di un'app web client-side + server-side («full stack») – II

3. L'utente interagisce con la pagina e genera **eventi utente**

Se l'evento utente è associato ad una funzione «di risposta» nell'applicazione client-side, la funzione viene eseguita, avviene cioè un'elaborazione. Come già detto l'elaborazione può avere uno (o entrambi) i seguenti effetti:

- Generare una richiesta di dati, o un invio di dati a fini di salvataggio/elaborazione, verso l'applicazione server-side (di cui l'applicazione client-side deve conoscere le opportune URL).
- Modificare il contenuto della pagina.

Per ora ci concentriamo su ciò che accade nel primo caso, vedremo dopo il secondo; è importante però ricordare appunto che potrebbero verificarsi **tutte e due** gli effetti in contemporanea (con le relative conseguenze, che vedremo a breve).

4. Il browser effettua una **richiesta HTTP** al Web Server, su una **URL appartenente all'API** dell'applicazione server-side

L'obiettivo della richiesta può essere di ottenere dei dati, di effettuare una elaborazione, e/o di renderli persistenti. Come già detto, la richiesta HTTP è non bloccante, e restituisce immediatamente il controllo all'applicazione client-side, che può proseguire con altre elaborazioni/richieste e restituire infine il controllo all'utente.

Le applicazioni client-side sono **single-thread** (esiste un modo in JavaScript di avviare thread in background, i **Web Workers**, ma è usato solo in casi particolari e non lo vedremo in questo corso) quindi è importante che le elaborazioni che avvengono in risposta ad eventi siano relativamente rapide, per non dare all'utente la sensazione di «interfaccia bloccata». È una rapidità relativa: ricordiamo sempre che i tempi dell'umano sono di diversi ordini di grandezza maggiori di quelli della macchina!

Esecuzione di un'app web client-side + server-side («full stack») – III

5. Il Web Server riceve la richiesta e la inoltra all'applicazione server-side.

L'applicazione server-side «spacchetta» la richiesta per capire qual è la funzionalità che il client desidera sia eseguita, e quali sono (se ci sono) i dati su cui operare. Deve dunque guardare il **method** della richiesta, prendere in considerazione il **path** all'interno della URL e l'eventuale **query string**, e verificare la presenza di un **body** contenente dei dati, controllandone la tipologia grazie al **Content-Type**.

Sulla base di queste informazioni, si attiva un'opportuna funzione che esegue l'elaborazione. Quasi sempre le elaborazioni dell'app server-side avvengono in interazione con un **Database Server** al fine di

- Ottenere informazioni da inviare al client, previa elaborazione delle stesse
- Memorizzare informazioni ricevute dal client o risultati di elaborazioni effettuate su di essi

Il risultato dell'elaborazione viene «confezionato» in una **Risposta HTTP**, in particolare andrà a definirne il **body**. Anche in questo caso il **Content-Type** dovrà riflettere la corretta tipologia di dato. Essa contiene anche le informazioni necessarie ad abbinarla alla Richiesta originale!

A differenza del client, il Web Server deve tenere conto del fatto che possono sopraggiungergli molte richieste in parallelo da client diversi. La maggior parte dei Web Server risponde a questa problematica **generando un thread di esecuzione separato** per ogni richiesta. Esiste tuttavia anche un approccio diverso (che è quello adottato da **Node.js**): le richieste vengono gestite in un **single threaded loop**, ma le elaborazioni conseguenti alle richieste sono realizzate in modo non-bloccante (**asincrono**). Vedremo questo aspetto meglio in seguito.

Esecuzione di un'app web client-side + server-side («full stack») - IV

6. La risposta HTTP viene inviata al client da cui si era originata la richiesta.

Il browser riceve la risposta HTTP che traduce in un opportuno **evento di rete** all'applicazione client-side che aveva originato la richiesta.

Come per gli eventi utente, l'ambiente di esecuzione in cui l'applicazione è in esecuzione richiama la funzione registrata per rispondere all'evento. Come vedremo in seguito, la registrazione di questa funzione avviene al momento della formulazione della richiesta HTTP.

Anche in questo caso, l'esecuzione della funzione può avere uno o entrambi i seguenti effetti:

- Generare un'altra richiesta all'applicazione server-side.
- Modificare il contenuto della pagina.

Esploriamo questa volta cosa accade nel secondo caso.

7. A fronte delle modifiche alla pagina, il browser **ri-renderizza** le parti della pagina che sono state modificate.

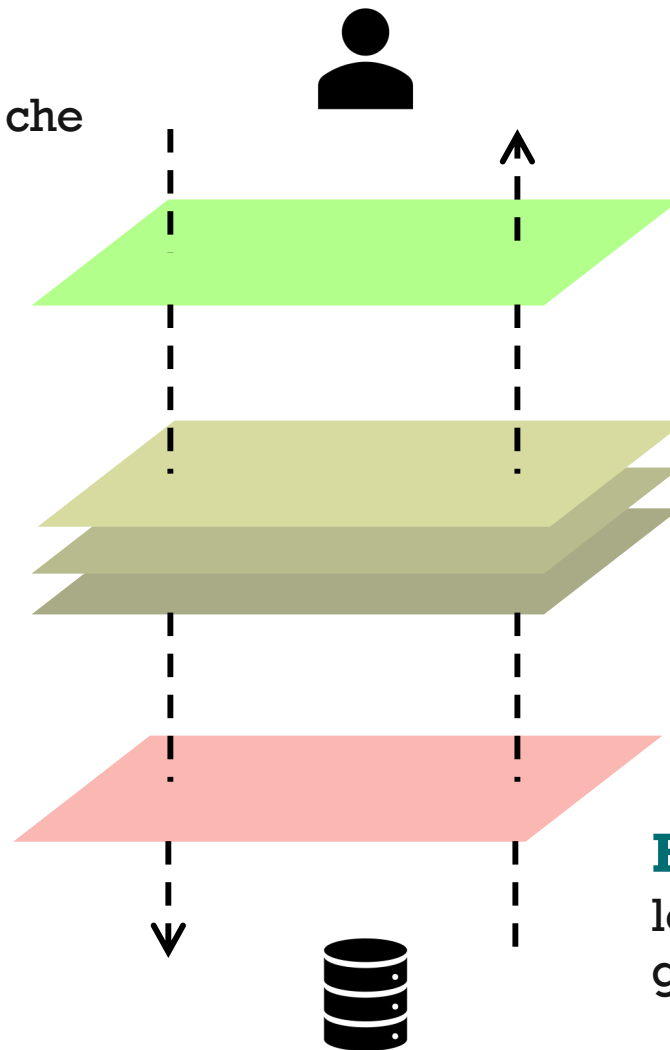
Il fatto che vengano ri-renderizzate solo le parti modificate è quello che rende l'interazione fluida dal punto di vista dell'utente. La ri-renderizzazione avviene non appena l'ambiente di esecuzione JavaScript rilascia il controllo sul thread di esecuzione: a quel punto infatti si può presumere che, almeno per il momento, le modifiche siano terminate e l'utente possa interagire con la «nuova» pagina.

Front-end e Back-end

La suddivisione logica delle applicazioni

Front-end:

lo **strato** di un'applicazione che interagisce con l'utente



Back-end:

lo **strato** di un'applicazione che gestisce la persistenza dei dati

Stratificazione logica di un'applicazione - I

Architettura a strati

L'architettura a strati è un pattern (modello, schema) di ideazione e progettazione di un'applicazione in cui la logica applicativa viene suddivisa su più **strati** o **livelli** (*layer*) ciascuno dei quali **offre funzionalità o servizi** al livello superiore **sfruttando le funzionalità o i servizi** del livello inferiore.

In questo approccio, che è molto diffuso per la sua modularità e versatilità, è centrale il concetto di **interfaccia** fra due strati: lo strato che **offre servizi/funzionalità** deve definire in modo formale le richieste che può accettare, con eventuali parametri (ossia l'**input**) e le risposte che il richiedente si può aspettare (ossia l'**output**).

Un vantaggio di questo approccio è quello che ogni strato può essere gestito separatamente: a patto di non modificare l'interfaccia verso lo strato superiore, è possibile modificare l'implementazione interna, correggere bug e rilasciare nuove versioni di uno strato senza alterare il funzionamento delle altre parti dell'applicazione.

Un altro vantaggio è che su di uno stesso strato si possono trovare più servizi/moduli diversi e alternativi, che tuttavia implementano la medesima interfaccia verso lo strato superiore. Lo strato superiore potrà utilizzare l'uno o l'altro senza dover modificare nulla del proprio comportamento.

Stratificazione logica di un'applicazione - II

Front-end e Back-end

Una suddivisione a strati molto comune e generale è quella che prevede:

- **Back-end** come livello più basso. È lo strato che gestisce la persistenza dei dati. Nelle applicazioni desktop solitamente si occupa di caricare/salvare file, cosa per cui si utilizzano ampiamente i servizi del file system e del sistema operativo: si tratta quindi di uno strato molto sottile. Nelle applicazioni web tipicamente interagisce con un **database server** e dunque è responsabile di tradurre i dati così come sono rappresentati nell'applicazione nel formato utilizzato dal database (ad es, record all'interno di una tabella in un DB relazionale), e viceversa. Potrà anche occuparsi di verifiche sui dati, controlli sulla correttezza delle transazioni ed eventuali rollback, ecc.
- **Front-end** come livello più alto. È lo strato che si occupa dell'interazione con l'utente. Di esso fanno parte la User Interface vera e propria (che è a tutti gli effetti l'interfaccia verso lo «strato superiore», che in questo caso è un essere umano!) ed eventuali moduli che elaborano le informazioni al fine di renderle fruibili.
- **Eventuali strati intermedi:** è possibile che ci siano strati intermedi che effettuano elaborazioni di dati (eventualmente, interagendo con servizi di terze parti) che non rientrano a pieno titolo né nel back-end né nel front-end; si tratta tipicamente di scelte architetturali nella progettazione di un'applicazione. Può avere senso effettuare un'elaborazione in uno strato intermedio se si vuole renderla indipendente sia dal front-end che dal back-end

Stratificazione logica di un'applicazione - III

Nelle applicazioni Web

- Il **front-end** si trova necessariamente lato client: è l'unico che può dotarsi di User Interface.
- Il **back-end** si trova tipicamente (ma non obbligatoriamente) lato server
- gli **eventuali strati intermedi** possono trovarsi sia lato client che lato server. Le elaborazioni lato client hanno il vantaggio di non sovraccaricare il server; tuttavia è possibile elaborare un dato lato client solo se le informazioni necessarie si trovano sul client stesso. Questo potrebbe non essere possibile per vari motivi: riservatezza delle informazioni in questione, non convenienza a trasferire grandi quantità di dati, ecc. Le elaborazioni lato server possono accedere a tutti i dati contenuti nel database; tuttavia se si tratta di elaborazioni costose in termini di risorse possono emergere problemi di scalabilità.

La configurazione più tipica, che è anche la configurazione di base, vede un'applicazione web composta da un front-end lato client e da un back-end lato server. Di conseguenza spesso «back-end» viene usato come sinonimo di «client-side», mentre «front-end» viene usato come sinonimo di «server-side». Sebbene tuttavia questi concetti siano spesso sovrapposti, è importante tenere presente che uno fa riferimento all'organizzazione logica dell'applicazione, l'altro alla sua organizzazione implementativa.